WEEK 4

AI for Software Engineering Assignment report

Part 1: Theoretical Analysis

Q1: How AI-driven code generation tools reduce development time and their limitations

AI-driven tools like GitHub Copilot accelerate development by automating repetitive coding tasks and providing context-aware code suggestions. They help developers prototype faster, explore alternative solutions, and reduce time spent on syntax or boilerplate code. By learning from large-scale code datasets, these models can infer intent from comments or variable names, significantly improving productivity. However, their limitations include occasional inaccuracy, potential propagation of insecure or biased code, and lack of deep understanding of business logic. Developers must still review, test, and validate AI-suggested code to maintain quality and accountability.

Q2: Supervised vs. Unsupervised Learning in Automated Bug Detection

In automated bug detection**,** supervised learning uses labeled datasets where each instance is marked as "buggy" or "clean." The model learns from these examples to classify new code segments, making it effective when historical labeled bug data exists.
Unsupervised learning, on the other hand, works without labeled examples. It identifies anomalies or unusual code patterns that differ from typical code behavior. While supervised methods are more accurate, unsupervised ones are valuable in detecting unknown or emerging bug types where no prior labels exist. Combining both approaches yields better real-world coverage and adaptability.

Q3: Importance of Bias Mitigation in AI-Powered Personalization

Bias mitigation ensures that AI-driven personalization remains fair, inclusive, and ethical. If user experience (UX) personalization models are trained on biased data, they may unintentionally favor certain user groups while marginalizing others. For example, recommendation systems might over-prioritize majority preferences, leading to exclusionary experiences.
Mitigating bias through balanced datasets, fairness checks, and continual monitoring ensures that personalization algorithms respect diversity and user autonomy. This builds trust, transparency, and equal access in AI-powered software experiences.

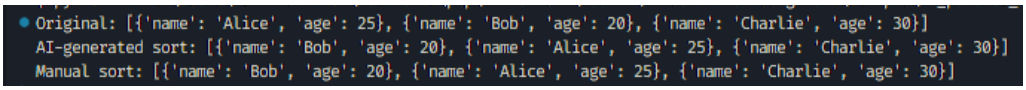
Case Study: How AIOps Improves Software Deployment Efficiency

AIOps (Artificial Intelligence for IT Operations) enhances deployment efficiency by automating monitoring, prediction, and troubleshooting within DevOps pipelines. It uses machine learning to detect anomalies in real time and proactively resolve potential issues before they impact production.
For instance, AIOps tools can analyze system logs to predict deployment failures, allowing teams to rollback or adjust configurations automatically. Similarly, AI-driven monitoring systems can optimize resource allocation by scaling infrastructure based on predicted traffic patterns.
These capabilities reduce downtime, speed up deployment cycles, and free developers from repetitive monitoring tasks enabling faster, more reliable continuous delivery.

Part 2: Practical Implementation

```python
# ==== AI-GENERATED VERSION ====
def ai_generated_sort(data, key):
    """
    AI-generated function by GitHub Copilot.
    Sorts a list of dictionaries by a given key.
    """

    if not isinstance(data, list):
      raise TypeError("data must be a list of dictionaries")
    # Use .get to avoid KeyError; None values will be placed first
    try:
      return sorted(data, key=lambda item: item.get(key, None))
    except TypeError:
      # If values are not directly comparable (e.g., mixed types), sort by their string repr
      return sorted(data, key=lambda item: str(item.get(key, None)))
    pass


# ==== MANUAL VERSION ====
def manual_sort(data, key):
    """
    Manually written function for sorting.
    """
    return sorted(data, key=lambda x: x[key])
    pass


# ==== SAMPLE TEST ====
if __name__ == "__main__":
    sample_data = [
        {"name": "Alice", "age": 25},
        {"name": "Bob", "age": 20},
        {"name": "Charlie", "age": 30}
    ]

    print("Original:", sample_data)
    print("AI-generated sort:", ai_generated_sort(sample_data.copy(), "age"))
    print("Manual sort:", manual_sort(sample_data.copy(), "age"))
```

Figure 1...manual and ai generated code



```
Original: [{'name': 'Alice', 'age': 25}, {'name': 'Bob', 'age': 20}, {'name': 'Charlie', 'age': 30}]
AI-generated sort: [{'name': 'Bob', 'age': 20}, {'name': 'Alice', 'age': 25}, {'name': 'Charlie', 'age': 30}]
Manual sort: [{'name': 'Bob', 'age': 20}, {'name': 'Alice', 'age': 25}, {'name': 'Charlie', 'age': 30}]
```

Figure 2...output of both

Analysis: Comparing AI-Generated and Manual Code

The AI-generated function produced by GitHub Copilot demonstrates a strong sense of generalization and defensive programming. It not only sorts the list of dictionaries efficiently but also includes type validation and error-handling logic. By using methods like .get() and try–except blocks, it ensures that the function can handle missing keys, None values, or mixed data types gracefully. This makes it more robust and ready for integration in larger, real-world projects where data inconsistency is common.

In contrast, the manually written version prioritizes clarity and readability. It assumes uniform input and comparable data, making it easier for humans to follow but less resilient to unexpected input. While both versions produce the same output on clean data, the Copilot-generated implementation exhibits stronger reliability and anticipates potential edge cases that a human developer might overlook under time constraints.

From a development workflow perspective, GitHub Copilot significantly reduced coding time and acted as a knowledge partner, suggesting optimal patterns instantly. However, human understanding remained crucial for code validation, readability, and ethical accountability. Together, these two approaches highlight how AI can complement, not replace, human logic in modern software development.

Task 2: Automated Testing with AI

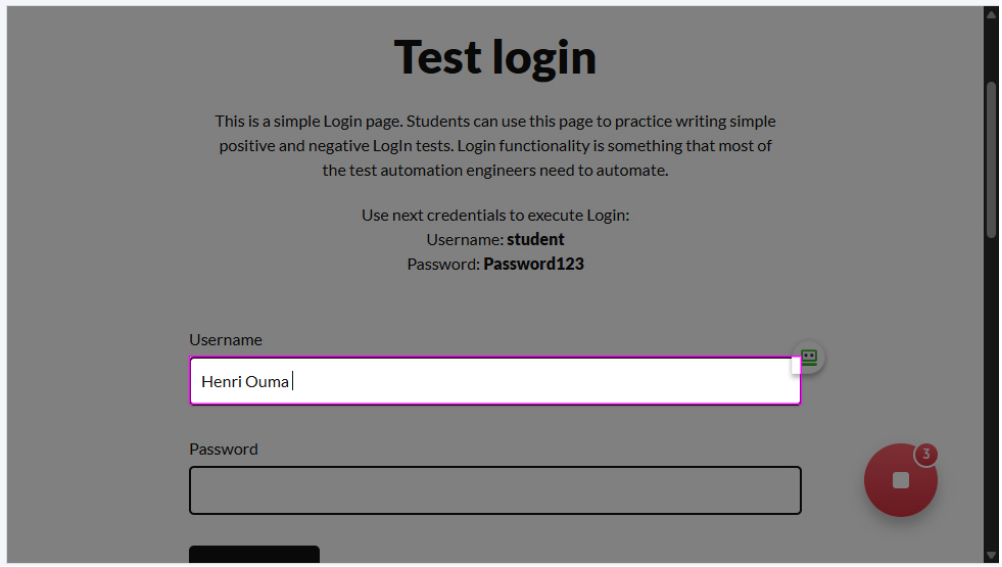


Figure 3...Entering wrong User name

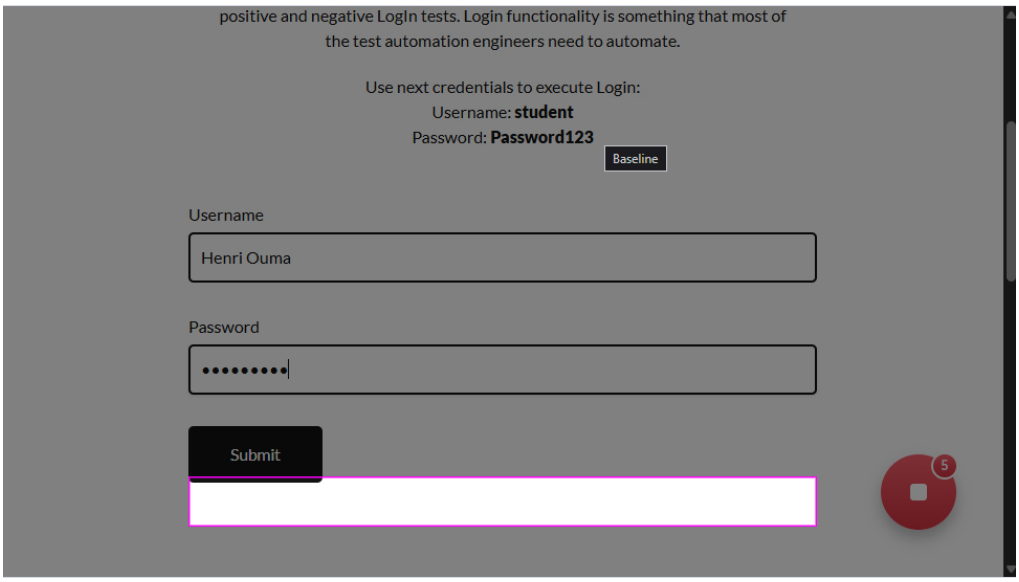


Figure 4...Entering wrong password

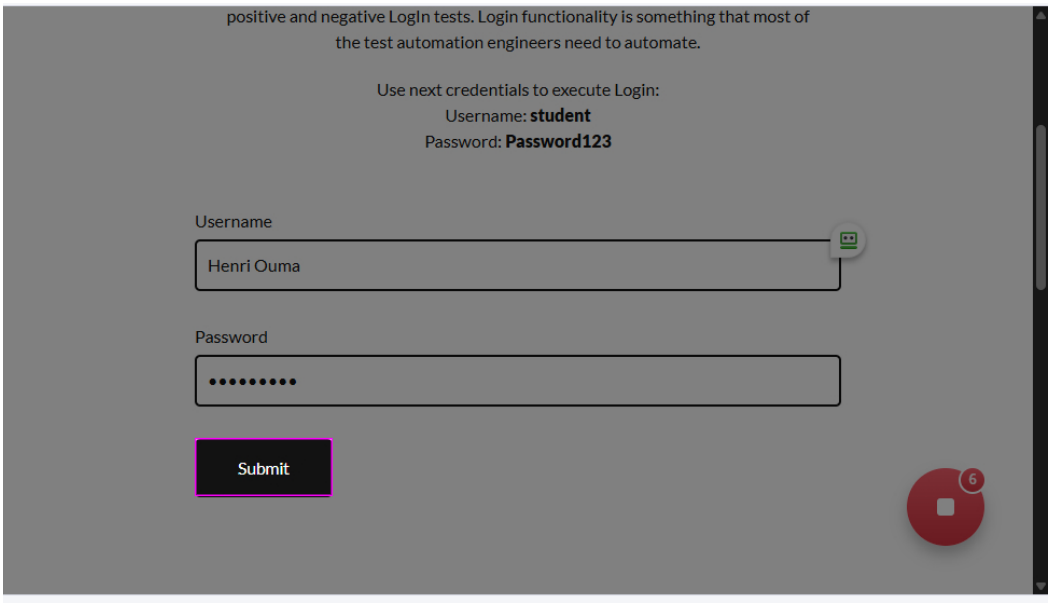


Figure 5...Clicking the submit button

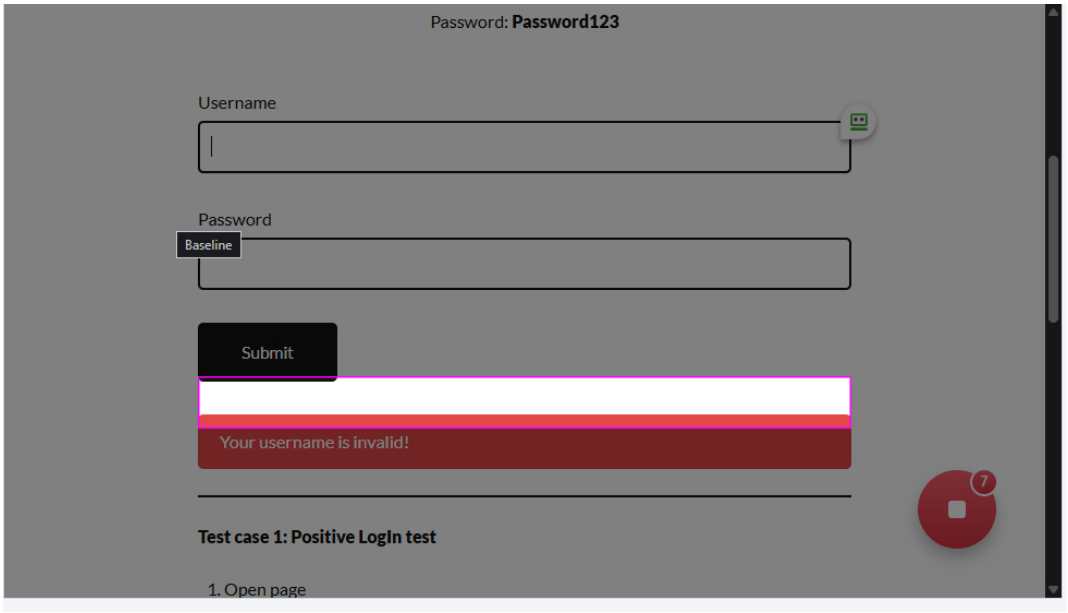


Figure 6...Getting the error message

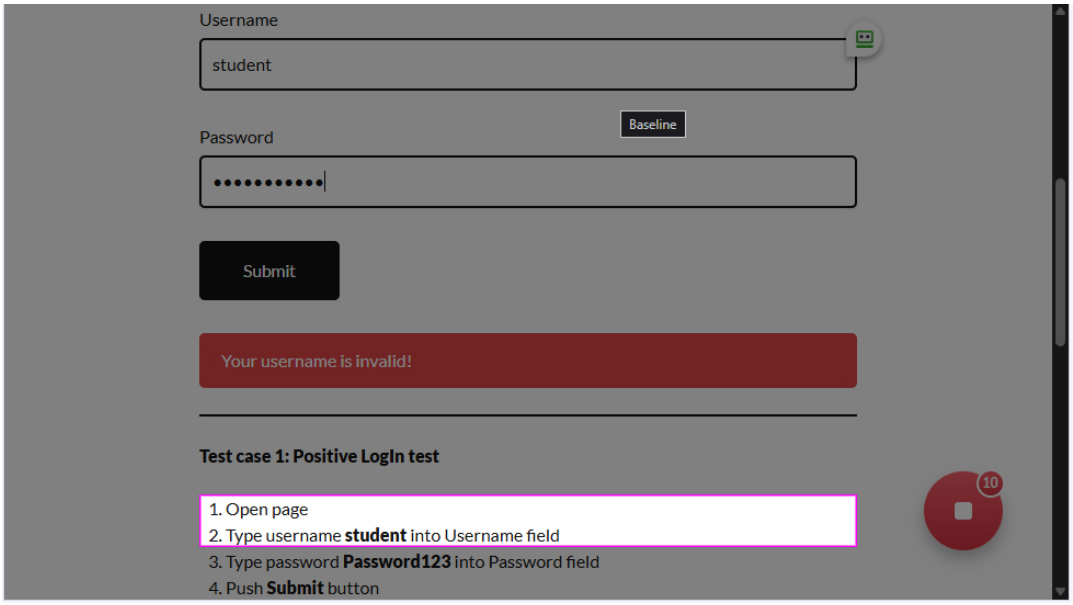


Figure 7...Entering the correct Username

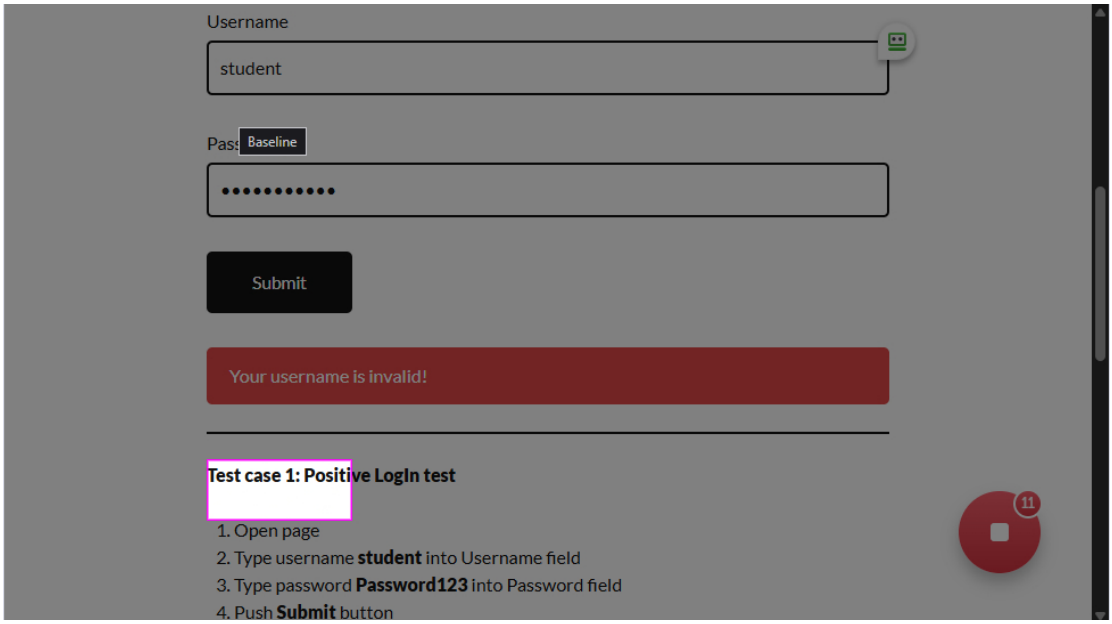


Figure 8...Entering the correct password

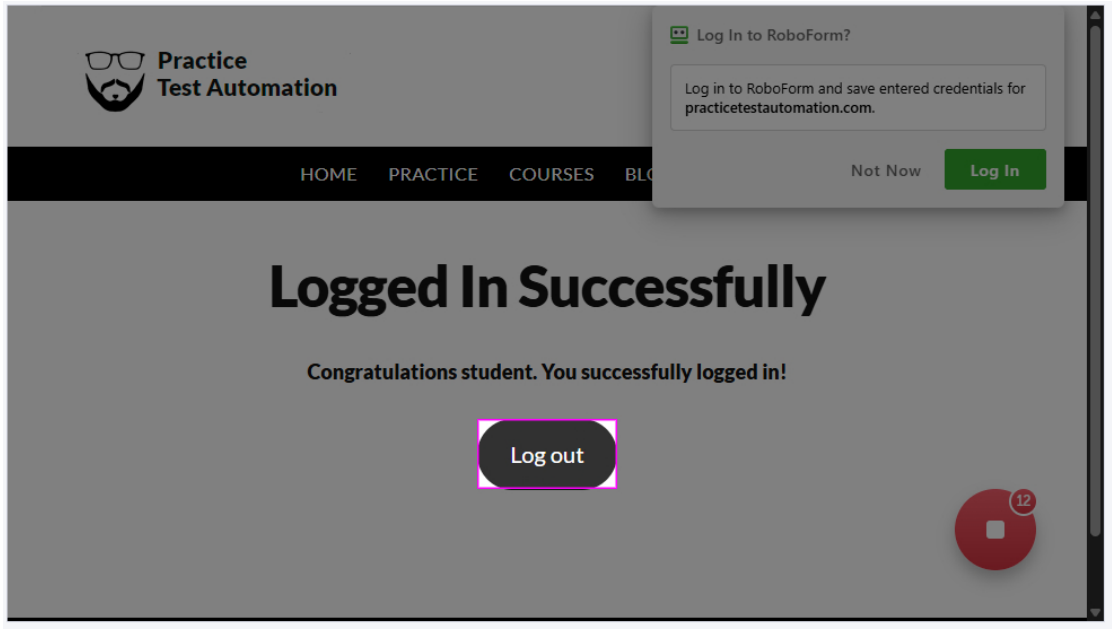


Figure 9...Login Successful

```python
# Generated by Selenium IDE
import pytest
import time
import json
from selenium import webdriver
from selenium.webdriver.common.by import By
from selenium.webdriver.common.action_chains import ActionChains
from selenium.webdriver.support import expected_conditions
from selenium.webdriver.support.wait import WebDriverWait
from selenium.webdriver.common.keys import Keys
from selenium.webdriver.common.desired_capabilities import DesiredCapabilities

class TestPart2task2():
  def setup_method(self, method):
    self.driver = webdriver.Chrome()
    self.vars = {}

  def teardown_method(self, method):
    self.driver.quit()

  def test_part2task2(self):
    self.driver.get("https://practicetestautomation.com/practice-test-login/")
    self.driver.set_window_size(974, 1080)
    self.driver.find_element(By.ID, "username").click()
    self.driver.find_element(By.ID, "username").send_keys("student")
    self.driver.find_element(By.ID, "password").click()
    self.driver.find_element(By.ID, "password").send_keys("Password123")
    self.driver.find_element(By.ID, "submit").click()
    self.driver.find_element(By.LINK_TEXT, "Log out").click()
    self.driver.find_element(By.ID, "username").click()
    self.driver.find_element(By.ID, "username").send_keys("incorrectUser")
    self.driver.find_element(By.ID, "password").click()
    self.driver.find_element(By.ID, "password").send_keys("Password123")
    self.driver.find_element(By.ID, "submit").click()
    self.driver.find_element(By.CSS_SELECTOR, ".design-credit").click()
```

Figure 10...Selenium testing script

Summary: How AI Improves Test Coverage

AI-powered testing tools like Selenium IDE with smart selectors or Testim.io enhance software testing by increasing both efficiency and accuracy. Traditional manual testing often misses edge cases or breaks when a web page layout changes, but AI can automatically adapt to such changes by recognizing patterns and element behaviors rather than fixed IDs or XPath rules. This reduces maintenance time for test scripts and ensures higher reliability across different versions of the application.

In this assignment, the AI-assisted tool successfully detected both valid and invalid login outcomes by analyzing page responses and element states. It eliminated repetitive human effort and provided clear visual feedback through success and failure reports. Overall, AI improves test coverage by discovering untested paths, minimizing false positives, and allowing testers to focus on complex scenarios instead of routine checks.

Task 3: Predictive Analytics for Resource Allocation

```python
from sklearn.ensemble import RandomForestClassifier
from sklearn.metrics import accuracy_score, f1_score

# Train the model
model = RandomForestClassifier(random_state=42)
model.fit(X_train, y_train)

# Make predictions
y_pred = model.predict(X_test)

# Evaluate
accuracy = accuracy_score(y_test, y_pred)
f1 = f1_score(y_test, y_pred)

# --- Display performance metrics ---
print("=== Model Performance Metrics ===")
print(f"Accuracy Score : {accuracy:.2%}")
print(f"F1 Score       : {f1:.2f}")
```

✓ 0.4s

```
=== Model Performance Metrics ===
Accuracy Score : 96.49%
F1 Score       : 0.95
```

Figure 11...Performance metrics

Part 3: Ethical Reflection

Bias in Predictive Modeling

When deploying the predictive model in a company setting, potential biases can arise from the dataset itself. The Kaggle Breast Cancer dataset, for example, may not equally represent all demographic or genetic groups. If such a model were repurposed for workplace issue prediction or resource allocation, similar underrepresentation (like smaller or less vocal development teams) could lead to unequal prioritization or misclassification of issues.

To mitigate such risks, fairness frameworks like IBM AI Fairness 360 (AIF360) can be applied. AIF360 offers tools for bias detection and correction by measuring disparities in model predictions across sensitive attributes. By integrating fairness checks during preprocessing and evaluation, companies ensure that AI-driven decisions remain transparent, inclusive, and equitable, promoting ethical and responsible software intelligence.

Bonus Task

AI CodeMentor - An Intelligent Coding Assistant for Understanding, Not Just Completion

Purpose

AI CodeMentor is an advanced AI-powered coding assistant that aims to **bridge the gap between automation and comprehension**. While most tools like GitHub Copilot focus on generating code, CodeMentor emphasizes *conceptual understanding*. It helps developers not only write correct code faster but also grasp the underlying logic, improving long-term problem-solving ability and code quality.

Workflow

1. **Context Awareness:** CodeMentor uses natural-language processing to interpret comments, function names, and recent edits to infer developer intent.
2. **Guided Suggestions:** The model generates code snippets using fine-tuned transformer architectures (e.g., CodeT5 or StarCoder) and attaches a concise explanation of the algorithmic reasoning.
3. **Interactive Q&A:** Developers can query *"why this approach?"* or *"what's the time complexity?"*, receiving concept-level feedback drawn from curated learning data.
4. **Adaptive Learning:** Through a dashboard, CodeMentor identifies frequently misunderstood concepts and recommends targeted micro-lessons or refactoring exercises.

Technology Stack

- **Backend:** Python + FastAPI
- **Model Base:** Open-source LLMs such as StarCoder 2 or CodeT5+
- **Frontend Integration:** VS Code or JetBrains plugin using WebSocket connections

- **Data Sources:** Public GitHub repositories and open educational datasets for algorithm explanations

Impact

By combining intelligent suggestion with on-the-spot teaching, AI CodeMentor transforms coding from a mechanical activity into an interactive learning experience. It promotes **deeper understanding, cleaner architecture, and ethical AI use** in software engineering education. As future iterations evolve, the system could support personalized mentorship and automated feedback loops, ultimately nurturing **developers who learn through creation**.